

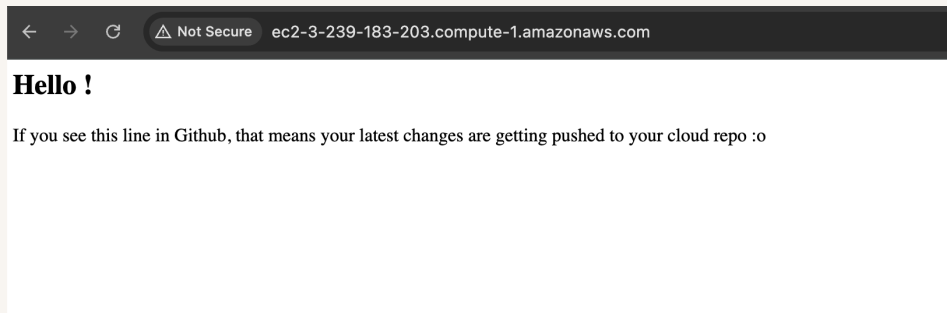


nextwork.org

Deploy a Web App with CodeDeploy



Arpita Chowdhury





Introducing Today's Project!

In this project, I will demonstrate how to provision a complete deployment environment using AWS CloudFormation, automate deployment actions with scripts, deploy a web application using AWS CodeDeploy, and safely roll back a deployment to recover from failures. I'm doing this project to learn how real-world teams automate application deployments, reduce manual errors, ensure repeatable infrastructure, and implement disaster recovery strategies that keep applications reliable and available in production.

Key tools and concepts

Services I used were Amazon EC2, AWS CodeBuild, AWS CodeDeploy, Amazon S3, AWS CodeArtifact, AWS IAM, and Amazon CloudWatch. Key concepts I learnt include CI/CD pipelines, build artifacts and revisions, in-place deployments, deployment lifecycle hooks, infrastructure permissions with IAM roles, automated dependency installation, reverse proxy configuration, deployment configurations, failure handling, and automatic rollback strategies for reliable production deployments.

Project reflection

deployment and connectivity issues, including fixing buildspec errors, resolving SSH and security group misconfigurations, and identifying why deployments succeeded while the application was not accessible. It was most rewarding to see the web application successfully deploy and load in the browser after implementing a full CI/CD pipeline with automated builds, deployments, and rollback handling, confirming that the entire process worked end to end.

This project is part five of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project Infrastructure as Code with CloudFormation.



Deployment Environment

To set up for CodeDeploy, I launched an EC2 instance and VPC because CodeDeploy needs a compute environment to deploy the application onto and a secure, isolated network to control traffic, permissions, and connectivity mirroring how applications are deployed safely in real production environments.

Instead of launching these resources manually, I used AWS CloudFormation to provision the EC2 instance, VPC, and related networking components in a repeatable and automated way using Infrastructure as Code. When I need to delete these resources, I can simply delete the CloudFormation stack, and AWS will automatically clean up all the associated resources together, saving time and preventing leftover infrastructure or unexpected costs.

Other resources created in this template include a VPC, public subnet, internet gateway, route table and route, security group, IAM roles and instance profile, and the EC2 instance itself. They're also in the template because CodeDeploy requires not just a server, but a fully functional and secure networking and permission setup, and defining everything in CloudFormation ensures the entire deployment environment is created consistently, automatically, and can be easily updated or deleted as a single stack.



CloudFormation > Stacks > NextWorkCodeDeployEC2Stack

Stacks (3)

Filter status: Active

View nested

Stacks

- NextWorkCodeDeployEC2Stack
2026-01-07 23:18:13 UTC-0600
CREATE_COMPLETE
- eksctl-nextwork-eks-cluster-cluster
2026-01-03 16:59:03 UTC-0600
CREATE_COMPLETE
- awsseb-e-vfzvfz9aqs-stack
2026-01-02 15:20:39 UTC-0600
UPDATE_COMPLETE

NextWorkCodeDeployEC2Stack

Delete Update stack Stack actions Create stack

Stack info Events Resources Outputs Parameters Template Change sets Git sync

Resources (11)

Search resources

Logical ID	Physical ID	Type	Status	Module
DeployRoleProfile	NextWorkCodeDeployEC2Stack-DeployRoleProfile-SKyqyuzKkrer	AWS::IAM::InstanceProfile	CREATE_IN_PROGRESS	-
InternetGateway	igw-067c6ff50d6a0e391	AWS::EC2::InternetGateway	CREATE_COMPLETE	-
PublicInternetRoute	rtb-01224043e1e3ea835j0.0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE	-
PublicRouteTable	rtb-01224043e1e3ea835	AWS::EC2::RouteTable	CREATE_COMPLETE	-
PublicSecurityGroup	sg-0f6b7132e72e6bbe5	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
PublicSubnetA	subnet-0506896ad4f8f3bb2	AWS::EC2::Subnet	CREATE_COMPLETE	-
PublicSubnetARouteTableAssociation	rtbassoc-042a096f4824d8cbf	AWS::EC2::SubnetRouteTableAssociation	CREATE_COMPLETE	-



Deployment Scripts

Scripts are executable files (usually .sh shell scripts) that run a series of commands automatically. They're used to make setup and deployment repeatable, consistent, and faster so you don't have to manually type the same commands every time. To set up CodeDeploy, I also wrote scripts to automate server configuration and deployment tasks like installing required dependencies (Tomcat and Apache/httpd), and configuring Apache as a reverse proxy so incoming traffic on port 80 is forwarded to my Tomcat web app running on port 8080. This ensures CodeDeploy can deploy updates reliably and the application can be accessed through a clean public URL.

The `install_dependencies.sh` will automatically set up everything my EC2 instance needs to run the application before CodeDeploy copies and starts the new version. In my case, it will: Install Tomcat (the Java application server that runs my .war web app) Install Apache HTTPD (the web server that receives traffic on port 80) Create/update an Apache configuration file (`/etc/httpd/conf.d/tomcat_manager.conf`) that routes requests on port 80 to Tomcat on port 8080, specifically to my app path (`/nextwork-web-project/`) So when CodeDeploy runs this script during deployment, it ensures the environment is prepared and consistent every time, even on a brand-new server.



The `start_server.sh` will start and enable the services required for my application to run and stay available after deployment. Specifically, it will: Start the Tomcat service, which runs my Java web application Enable Tomcat so it automatically starts on system reboot Start the Apache HTTPD service, which handles incoming web traffic on port 80 Enable Apache HTTPD so it also restarts automatically if the server reboots This ensures that after CodeDeploy finishes deploying the application, the web server and application server are both running and ready to serve users without any manual intervention.

The `stop_server.sh` will safely shut down any running web services before a new deployment starts. It works by: Checking whether Apache HTTPD is currently running using `pgrep` Stopping the Apache service only if it is already running (preventing unnecessary errors) Checking whether Tomcat is running Stopping the Tomcat service if it is active This is important in CodeDeploy because stopping existing services first prevents file locks, port conflicts, and inconsistent states, ensuring a clean and reliable deployment of the new application version.



appspec.yml

I wrote the appspec.yml file to define how CodeDeploy deploys my application end to end. It specifies where the application artifact is copied, and which scripts run before stopping services, installing dependencies, and starting the server again. The key sections include the files block (for deploying the WAR file) and the hooks block (for running install, stop, and start scripts), ensuring a clean, automated, and reliable deployment process.

I also updated buildspec.yml because CodeDeploy needs all deployment files packaged together in the build artifact. By including the compiled WAR file, the appspec.yml, and all deployment scripts, CodeBuild produces a complete, deployable bundle that CodeDeploy can use directly without any manual steps.

```
! appspec.yml
1  version: 0.0
2  os: linux
3  files:
4    - source: /target/nextwork-web-project.war
5      destination: /usr/share/tomcat/webapps/
6  hooks:
7    BeforeInstall:
8      - location: scripts/install_dependencies.sh
9        timeout: 300
10       runas: root
11    ApplicationStart:
12      - location: scripts/start_server.sh
13        timeout: 300
14        runas: root
15    ApplicationStop:
16      - location: scripts/stop_server.sh
17        timeout: 300
18        runas: root
```



Setting Up CodeDeploy

A deployment group is a logical set of target resources (such as EC2 instances or Auto Scaling groups) that CodeDeploy deploys an application to. It controls where and how deployments run, including settings like deployment strategy, rollback behavior, and instance selection (via tags or ASGs). A CodeDeploy application is a container that represents the software you want to deploy. It defines the compute platform (EC2/On-Prem, Lambda, or ECS) and acts as the top-level object that links your application revisions (artifacts), deployment groups, and deployment history together.

To set up a deployment group, you also need to create an IAM role to grant AWS CodeDeploy permission to interact with other AWS services, such as identifying target EC2 instances, accessing deployment artifacts from Amazon S3, and managing deployment lifecycle operations securely on your behalf.

Tags are helpful for organizing, identifying, and managing AWS resources at scale especially for filtering resources, controlling access, tracking costs, and targeting services like CodeDeploy. I used the tag (for example, Name: NextWorkCodeDeploy or Environment: Dev) to clearly identify this EC2 instance and allow the deployment group to select the correct instance automatically during deployments.



Instances (1) [Info](#)

Last updated less than a minute ago

Connect

Instance state ▾

Actions ▾

Launch instances ▾

Instance ID = i-Ofd1e2d2a359a0643

Clear filters

< 1 >

☐	Name	Instance ID	Instance state ▾	Instance type ▾	Status check	Alarm status	Availability Zone ▾	Public IPv4 DNS ▾
☐	NextWorkCod...	i-Ofd1e2d2a359a0643	Running	t2.micro	2/2 checks passed View alarms +		us-east-1a	ec2-3-239-183-203.co...



Deployment configurations

Another key setting is the deployment configuration, which affects how traffic is shifted and how instances are updated during a deployment. I used `CodeDeployDefault.AllAtOnce`, so the new application version is deployed to all instances at the same time, making the deployment fast and simple, which is ideal for a small environment or a single-instance setup.

In order to connect AWS CodeDeploy to my EC2 instance, a CodeDeploy Agent is also set up to run on the instance to listen for deployment instructions from CodeDeploy, pull the application bundle from Amazon S3, execute the lifecycle scripts defined in `appspec.yml`, and report deployment status back to the CodeDeploy service.

The screenshot shows the 'Deployment settings' form in the AWS CodeDeploy console. It is divided into two main sections: 'Deployment configuration' and 'Load balancer'. In the 'Deployment configuration' section, a dropdown menu is set to 'CodeDeployDefault.AllAtOnce', with a 'Create deployment configuration' button to its right. The 'Load balancer' section includes a description, a checked 'Enable load balancing' checkbox, and a 'Load balancer type' section with two radio button options: 'Application Load Balancer or Network Load Balancer' (selected) and 'Classic Load Balancer'. At the bottom, there is a 'Cancel' button and an orange 'Create deployment group' button.

Deployment settings

Deployment configuration
Choose from a list of default and custom deployment configurations. A deployment configuration is a set of rules that determines how fast an application is deployed and the success or failure conditions for a deployment.

CodeDeployDefault.AllAtOnce ▼ or Create deployment configuration

Load balancer

Select a load balancer to manage incoming traffic during the deployment process. The load balancer blocks traffic from each instance while it's being deployed to and allows traffic to it again after the deployment succeeds.

☒ Enable load balancing

Load balancer type

☒ Application Load Balancer or Network Load Balancer

☐ Classic Load Balancer

► Advanced - optional

Cancel Create deployment group



Success!

A CodeDeploy deployment is a single execution that takes a specific application revision (artifact) and deploys it to the target compute resources. The difference is that a deployment group defines where and how deployments run (target EC2 instances, IAM role, deployment configuration), while a deployment is the actual action that uses that deployment group to push and install the application.

I had to configure a revision location, which means specifying where CodeDeploy can find the packaged application files to deploy. My revision location was an Amazon S3 bucket (`s3://nextwork-devops-cicd-arpita/nextwork-devops-cicd-artifact`) containing a .zip file, which holds the application artifact, `appspec.yml`, and deployment scripts used during the deployment.

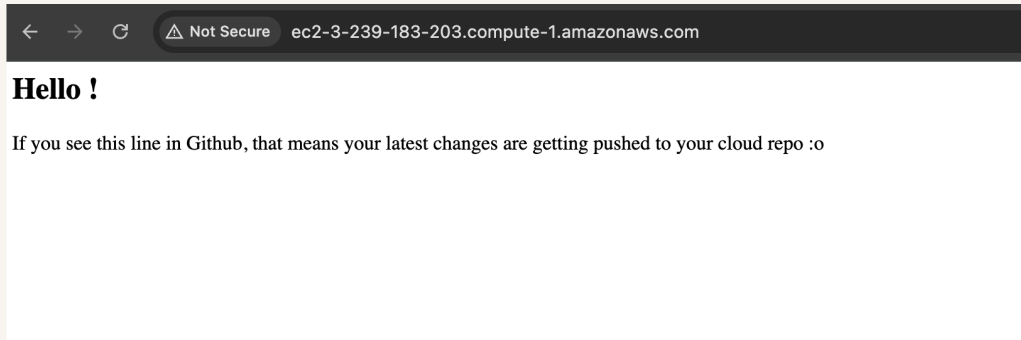
To check that the deployment was a success, I visited the EC2 instance's public URL (`http://ec2-3-239-183-203.compute-1.amazonaws.com`). I saw my web page load with the "Hello!" message and the confirmation text indicating my latest GitHub changes were successfully deployed to the server—proving the CodeDeploy deployment worked and the app is now live.



Arpita Chowdhury

NextWork Student

nextwork.org





Disaster Recovery

In a project extension, I decided to intentionally test CodeDeploy's failure and rollback behavior by introducing an error into one of the deployment lifecycle scripts. The intentional error I created was misspelling the `systemctl` command as `systemctll` in the `stop_server.sh` script and forcing the script to exit with a non-zero status. This will cause the deployment to fail because CodeDeploy executes this script during the `ApplicationStop` hook, and when the command fails, the script exits with an error, which CodeDeploy detects and marks the deployment as failed.

I also enabled rollbacks with this deployment, which means that if the deployment fails at any stage, CodeDeploy will automatically stop the failed deployment and revert the EC2 instance back to the last known successful version of the application, ensuring minimal downtime and preventing broken or partially deployed code from remaining live.

When my deployment failed, the automatic rollback reverted the EC2 instance back to the last successful application version because CodeDeploy detected a failure during the deployment lifecycle and was configured to restore service automatically. To actually recover from the issue long term, I'd have to identify and fix the root cause in the deployment scripts or application code, generate a new build artifact, and redeploy it successfully. In production environments, I would implement automatic rollbacks using safer deployment strategies like rolling or blue/green deployments, attach rollbacks to real health checks and CloudWatch alarms, run post-deployment validation tests, and maintain strong monitoring and versioned artifacts so failures are detected early and service availability is preserved with minimal impact.



Arpita Chowdhury

NextWork Student

nextwork.org

	Deployment ID	Status	Deploym...	Compute platform	Application	Deploym...	Revision L...	Initiating event
☐	d-PIYH6AZ6N	 Failed	In place	EC2/on-premises	nextwork-...	nextwork-...	s3://next...	CodeDeploy rollback
☐	d-VK66QY6JA	 Failed	In place	EC2/on-premises	nextwork-...	nextwork-...	s3://next...	User action



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

